

Linters for Python Code Quality – Reflection

PyLint Output (excerpt):

```
***** Module code_lint
code_lint.py:27:0: W0301: Unnecessary semicolon (unnecessary-semicolon)
code_lint.py:31:0: C0325: Unnecessary parens after 'return' keyword
(superfluous-parens)
code_lint.py:35:0: C0115: Missing class docstring (missing-class-
docstring)
code_lint.py:1:0: C0114: Missing module docstring (missing-module-
docstring)
...
Your code has been rated at 0.00/10
```

Pyflakes Output (excerpt):

```
code_lint.py:13:5: local variable 'some_global_var' is assigned to but
never used
code_lint.py:40:44: 'pi' may be undefined, or defined from star imports:
math
code_lint.py:41:16: local variable 'time' defined in enclosing scope on
line 5 referenced before assignment
```

Pycodestyle Output (excerpt):

```
code_lint.py:14:15: E225 missing whitespace around operator
code_lint.py:27:25: E703 statement ends with a semicolon
code_lint.py:31:23: E275 missing whitespace after keyword
code_lint.py:40:80: E501 line too long (83 > 79 characters)
```

Pydocstyle Output (excerpt):

```
code_lint.py:1 at module level: D100: Missing docstring in public module
code_lint.py:10 in public function `multiply`: D400: First line should
end with a period
code_lint.py:20 in public function `is_sum_lucky`: D205: 1 blank line
required between summary line and description
code_lint.py:35 in public class `SomeClass`: D101: Missing docstring in
public class
```

Reflection

Running the Python code through four different linters provided a clear view of its strengths and weaknesses and highlighted how each tool targets specific aspects of code quality.

Pylint was the most comprehensive, flagging style issues, coding errors, and documentation gaps. It identified unused variables, unreachable code, redefined names, and missing docstrings, ultimately giving the file a score of 0/10. This strictness is valuable for ensuring a high standard of overall code quality but can be overwhelming for beginner programmers due to the sheer number of warnings.

Pyflakes, in contrast, focused solely on coding errors like undefined variables and unused assignments. It ignored style and docstring issues entirely, demonstrating that it is lightweight and fast, making it effective for quickly detecting potential runtime problems.

Pycodestyle was useful for enforcing PEP8 formatting rules, including whitespace, semicolons, and line lengths. While it does not detect logic or runtime issues, it is essential for maintaining consistent code style across a project.

Pydocstyle concentrated on documentation quality, highlighting missing docstrings and structural issues within the documentation. It is particularly useful in professional or collaborative environments where clear and consistent documentation is critical.

In conclusion, no single linter addresses all aspects of code quality. Combining multiple tools provides a holistic view: Pylint for overall quality, Pyflakes for logic errors, Pycodestyle for formatting, and Pydocstyle for documentation. This layered approach ensures that code is correct, readable, maintainable, and well-documented, which aligns with modern software quality standards.